

BSK PG Trusted Third Party Demo

Generated by Doxygen 1.9.8

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 apps/client/src/api.ts File Reference	3
2.1.1 Function Documentation	3
2.1.1.1 encryptForTtp()	3
2.1.1.2 loadTtpPublicKey()	4
2.1.2 Variable Documentation	4
2.1.2.1 service	4
2.1.2.2 serviceBaseUrl	4
2.1.2.3 serviceQuery	4
2.1.2.4 ttp	4
2.1.2.5 ttpBaseUrl	4
2.1.2.6 ttpProtocol	4
2.1.2.7 ttpQuery	5
2.1.2.8 type	5
2.2 apps/client/src/components/evidence.tsx File Reference	5
2.2.1 Function Documentation	5
2.2.1.1 Evidence()	5
2.3 apps/client/src/components/utlis.ts File Reference	5
2.3.1 Function Documentation	5
2.3.1.1 cn()	5
2.4 apps/client/src/lib/client-security-state.ts File Reference	5
2.4.1 Variable Documentation	6
2.4.1.1 ActiveSession	6
2.4.1.2 clientSecurityState	6
2.4.1.3 current	6
2.4.1.4 RegisteredUser	6
2.5 apps/client/src/lib/security-flow.ts File Reference	6
2.5.1 Function Documentation	7
2.5.1.1 activeSession()	7
2.5.1.2 createRogueUserCertificate()	7
2.5.1.3 loadRegisteredServer()	7
2.5.1.4 registerUserIdentity()	7
2.5.2 Variable Documentation	7
2.5.2.1 DemoFileCurrentResponse	7
2.5.2.2 DemoFileServiceRequest	8
2.5.2.3 DemoFileServiceResponse	8
2.5.2.4 DemoFileTransferInput	8
2.5.2.5 RegisteredServer	8
2.5.2.6 securityFlow	8

2.5.2.7 ServerDemoFileCurrentResponse	8
2.6 apps/client/src/lib/useSecurityFlow.ts File Reference	9
2.6.1 Function Documentation	9
2.6.1.1 useSecurityFlow()	9
2.6.2 Variable Documentation	9
2.6.2.1 CLIENT_STATUS_KEY	9
2.7 apps/client/src/routes/index.tsx File Reference	9
2.7.1 Function Documentation	10
2.7.1.1 base64ByteLength()	10
2.7.1.2 base64ToText()	10
2.7.1.3 EncryptedPayloadView()	10
2.7.1.4 isTextFile()	10
2.7.1.5 readFileAsBase64()	10
2.7.1.6 SecurityFlowPage()	10
2.7.1.7 TransferredFilePreview()	11
2.7.2 Variable Documentation	11
2.7.2.1 FileServiceId	11
2.7.2.2 Route	11
2.8 apps/tp/src/contract.ts File Reference	11
2.8.1 Function Documentation	11
2.8.1.1 serverAuthenticationPayload()	11
2.8.1.2 userAuthenticationPayload()	12
2.8.2 Variable Documentation	12
2.8.2.1 IdentityPublicKeys	12
2.8.2.2 RegisterIdentityInput	12
2.8.2.3 Role	12
2.8.2.4 ServerAuthenticationInput	13
2.8.2.5 ServerSessionKeyInput	13
2.8.2.6 UserAuthenticationInput	13
2.8.2.7 UserAuthenticationRequest	13
2.8.2.8 UserAuthRedirectInput	13
2.9 apps/server/src/protocol.ts File Reference	14
2.9.1 Function Documentation	14
2.9.1.1 closeLocalSession()	14
2.9.1.2 exchangeProtectedServiceData()	14
2.9.1.3 fetchServerSessionKey()	15
2.9.1.4 handleFileServiceRequest()	15
2.9.1.5 registerProtectedServer()	15
2.9.1.6 requestService()	15
2.9.2 Variable Documentation	16
2.9.2.1 DemoFileEmptyResponse	16
2.9.2.2 DemoFileResponse	16

2.9.2.3 DemoFileServiceRequest	16
2.9.2.4 DemoFileServiceResponse	16
2.9.2.5 ttp	16
2.9.2.6 ttpBaseUrl	17
2.9.2.7 UploadFileRequest	17
2.10 apps/ttp/src/protocol.ts File Reference	17
2.10.1 Function Documentation	17
2.10.1.1 authenticateServerCertificate()	17
2.10.1.2 authenticateUserForServer()	18
2.10.1.3 closeSession()	18
2.10.1.4 decryptUserAuthenticationRequest()	18
2.10.1.5 registerIdentity()	19
2.10.1.6 requestUserAuthentication()	19
2.10.1.7 serverSessionKey()	19
2.10.1.8 ttpPublicKeyResponse()	19
2.10.2 Variable Documentation	20
2.10.2.1 CertifiableIdentity	20
2.10.2.2 CertificateClaim	20
2.11 apps/server/src/rpc.ts File Reference	20
2.11.1 Function Documentation	20
2.11.1.1 reject()	20
2.11.2 Variable Documentation	20
2.11.2.1 ServiceRouter	20
2.12 apps/ttp/src/rpc.ts File Reference	21
2.12.1 Function Documentation	21
2.12.1.1 reject()	21
2.12.2 Variable Documentation	21
2.12.2.1 TtpRouter	21
2.13 apps/server/src/state.ts File Reference	21
2.13.1 Function Documentation	22
2.13.1.1 log()	22
2.13.1.2 readServiceServerStatus()	22
2.13.1.3 resetServiceServerState()	22
2.13.2 Variable Documentation	22
2.13.2.1 DemoFileMeta	22
2.13.2.2 DemoStoredFile	22
2.13.2.3 logger	23
2.13.2.4 ServiceServerState	23
2.13.2.5 ServiceServerStatus	23
2.13.2.6 state	23
2.14 apps/ttp/src/state.ts File Reference	23
2.14.1 Variable Documentation	24

2.14.1.1 ca	24
2.14.1.2 IdentityRecord	24
2.14.1.3 log	24
2.14.1.4 logger	24
2.14.1.5 PendingAuthRecord	24
2.14.1.6 pendingAuths	24
2.14.1.7 registeredIdentities	24
2.14.1.8 SessionRecord	24
2.14.1.9 sessions	25
2.14.1.10 sessionsByRequest	25
2.15 packages/crypto/src/certificates.ts File Reference	25
2.15.1 Function Documentation	25
2.15.1.1 createCertificateAuthority()	25
2.15.2 Variable Documentation	26
2.15.2.1 DAY_MS	26
2.16 packages/crypto/src/crypto.ts File Reference	26
2.16.1 Function Documentation	27
2.16.1.1 encodeHybridPayloadBase64()	27
2.16.1.2 encryptRsaOaepSha256()	27
2.16.1.3 rsaOaepSha256()	27
2.16.2 Variable Documentation	27
2.16.2.1 AES_256_KEY_BYTES	27
2.16.2.2 AES_GCM_IV_BYTES	27
2.16.2.3 AES_GCM_TAG_BITS	27
2.16.2.4 aesGcm	27
2.16.2.5 random	28
2.16.2.6 rsa	28
2.16.2.7 RSA_BITS	28
2.16.2.8 signedPayload	28
2.17 packages/crypto/src/types.ts File Reference	28
2.17.1 Variable Documentation	29
2.17.1.1 AesGcmPayload	29
2.17.1.2 CertificateAuthority	29
2.17.1.3 CertificateAuthorityOptions	29
2.17.1.4 HybridEncryptedPayload	30
2.17.1.5 IdentityCertificateInput	30
2.17.1.6 RsaDecryptor	30
2.17.1.7 RsaEncryptor	30
2.17.1.8 RsaPair	30
2.17.1.9 SessionEncryptedPayload	31
2.17.1.10 SessionTicket	31
2.18 packages/rpc/src/client.ts File Reference	31

2.19 packages/rpc/src/log.ts File Reference	31
2.19.1 Function Documentation	32
2.19.1.1 createSecurityLogger()	32
2.19.1.2 logDirectory()	32
2.19.1.3 logPath()	32
2.19.2 Variable Documentation	32
2.19.2.1 DEFAULT_LOG_DIR	32
2.19.2.2 SecurityLogActor	32
2.19.2.3 SecurityLogLevel	32
2.20 packages/rpc/src/server.ts File Reference	32
2.20.1 Function Documentation	32
2.20.1.1 createRpcFetch()	32
Index	33

Chapter 1

File Index

1.1 File List

Here is a list of all files with brief descriptions:

apps/client/src/ api.ts	3
apps/client/src/components/ evidence.tsx	5
apps/client/src/components/ utils.ts	5
apps/client/src/lib/ client-security-state.ts	5
apps/client/src/lib/ security-flow.ts	6
apps/client/src/lib/ useSecurityFlow.ts	9
apps/client/src/routes/ index.tsx	9
apps/server/src/ protocol.ts	14
apps/server/src/ rpc.ts	20
apps/server/src/ state.ts	21
apps/ttp/src/ contract.ts	11
apps/ttp/src/ protocol.ts	17
apps/ttp/src/ rpc.ts	21
apps/ttp/src/ state.ts	23
packages/crypto/src/ certificates.ts	25
packages/crypto/src/ crypto.ts	26
packages/crypto/src/ types.ts	28
packages/rpc/src/ client.ts	31
packages/rpc/src/ log.ts	31
packages/rpc/src/ server.ts	32

Chapter 2

File Documentation

2.1 apps/client/src/api.ts File Reference

```
import rsa;
import "@bsk/crypto";
import "@bsk/rpc/client";
import "@orpc/tanstack-query";
import "server";
import "ttp";
import "ttp/contract";
```

Functions

- function async [loadTtpPublicKey](#) ()
- function async [encryptForTtp](#) (plaintext:string)

Variables

- const [ttpBaseUrl](#) = import.meta.env.VITE_TTP_API_BASE_URL ?? "http://localhost:3001"
- const [serviceBaseUrl](#) = import.meta.env.VITE_SERVICE_API_BASE_URL ?? "http://localhost:3002"
- export const [ttp](#) = createRpcClient<[TtpRouter](#)>([ttpBaseUrl](#))
- export const [service](#) = createRpcClient<[ServiceRouter](#)>([serviceBaseUrl](#))
- export const [ttpQuery](#) = createTanstackQueryUtils([ttp](#))
- export const [serviceQuery](#) = createTanstackQueryUtils([service](#))
- export type { [UserAuthenticationRequest](#) }
- export const [ttpProtocol](#)

2.1.1 Function Documentation

2.1.1.1 [encryptForTtp\(\)](#)

```
function async encryptForTtp (  
    plaintext:string )
```

2.1.1.2 loadTtpPublicKey()

```
function async loadTtpPublicKey ( )
```

2.1.2 Variable Documentation

2.1.2.1 service

```
export const service = createRpcClient<ServiceRouter>(serviceBaseUrl)
```

2.1.2.2 serviceBaseUrl

```
const serviceBaseUrl = import.meta.env.VITE_SERVICE_API_BASE_URL ?? "http://localhost:3002"
```

2.1.2.3 serviceQuery

```
export const serviceQuery = createTanstackQueryUtils(service)
```

2.1.2.4 ttp

```
export const ttp = createRpcClient<TtpRouter>(ttpBaseUrl)
```

2.1.2.5 ttpBaseUrl

```
const ttpBaseUrl = import.meta.env.VITE_TTP_API_BASE_URL ?? "http://localhost:3001"
```

2.1.2.6 ttpProtocol

```
export const ttpProtocol
```

Initial value:

```
= {
  async registerUser(id: string, publicKeys: IdentityPublicKeys) {
    return ttp.register({
      role: "user",
      encryptedId: await encryptForTtp(id),
      publicKeys,
    });
  },

  requestUserAuthentication(requestId: string) {
    return ttp.auth.redirect({ requestId });
  },

  async authenticateUser(request: UserAuthenticationRequest) {
    return ttp.auth.user({
      encryptedAuthMaterial: await encryptForTtp(JSON.stringify(request)),
    });
  },

  closeSession(sessionId: string) {
    return ttp.session.close({ sessionId });
  },
}
```

2.1.2.7 ttpQuery

```
export const ttpQuery = createTanstackQueryUtils(ttp)
```

2.1.2.8 type

```
export type { UserAuthenticationRequest }
```

2.2 apps/client/src/components/evidence.tsx File Reference

Functions

- function export [Evidence](#) ({ label, value }: { label:string;value?:string })

2.2.1 Function Documentation

2.2.1.1 Evidence()

```
function export Evidence (  
  { label, value }: { label:string;value?:string } )
```

2.3 apps/client/src/components/utils.ts File Reference

```
import type;  
import "clsx";  
import "tailwind-merge";
```

Functions

- function export [cn](#) (...inputs:ClassValue[])

2.3.1 Function Documentation

2.3.1.1 cn()

```
function export cn (  
  ...inputs:ClassValue[] )
```

2.4 apps/client/src/lib/client-security-state.ts File Reference

```
import hash;  
import "@bsk/crypto";
```

Variables

- export type [RegisteredUser](#)
- type [ActiveSession](#)
- const [current](#)
- export const [clientSecurityState](#)

2.4.1 Variable Documentation

2.4.1.1 ActiveSession

```
type ActiveSession
```

Initial value:

```
= {  
  sessionId: string;  
  userSessionKey: string;  
}
```

2.4.1.2 clientSecurityState

```
export const clientSecurityState
```

2.4.1.3 current

```
const current
```

2.4.1.4 RegisteredUser

```
export type RegisteredUser
```

Initial value:

```
= {  
  id: string;  
  authKeyPair: RsaPair;  
  exchangeKeyPair: RsaPair;  
  certificatePem: string;  
}
```

2.5 apps/client/src/lib/security-flow.ts File Reference

```
import aesGcm;  
import "@bsk/crypto";  
import "ttp/contract";  
import "#/api";  
import "../client-security-state";
```

Functions

- function async [registerUserIdentity](#) ()
- function async [createRogueUserCertificate](#) (user:RegisteredUser)
- function [activeSession](#) ()
- function async [loadRegisteredServer](#) ()

Variables

- type [RegisteredServer](#)
- type [DemoFileServiceRequest](#)
- export type [DemoFileTransferInput](#)
- export type [DemoFileCurrentResponse](#)
- type [ServerDemoFileCurrentResponse](#) = Omit<[DemoFileCurrentResponse](#), "encryptedPayload">
- export type [DemoFileServiceResponse](#)
- export const [securityFlow](#)

2.5.1 Function Documentation

2.5.1.1 activeSession()

```
function activeSession ( )
```

2.5.1.2 createRogueUserCertificate()

```
function async createRogueUserCertificate (
    user:RegisteredUser )
```

2.5.1.3 loadRegisteredServer()

```
function async loadRegisteredServer ( )
```

2.5.1.4 registerUserIdentity()

```
function async registerUserIdentity ( )
```

2.5.2 Variable Documentation

2.5.2.1 DemoFileCurrentResponse

```
export type DemoFileCurrentResponse
```

Initial value:

```
= {
  kind: "file.current";
  name: string;
  mimeType: string;
  size: number;
  contentBase64: string;
  storedAt: string;
  encryptedPayload: SessionEncryptedPayload;
}
```

2.5.2.2 DemoFileServiceRequest

```
type DemoFileServiceRequest
```

Initial value:

```
=  
| {  
  kind: "file.upload";  
  name: string;  
  mimeType: string;  
  contentBase64: string;  
}  
| {  
  kind: "file.view";  
}
```

2.5.2.3 DemoFileServiceResponse

```
export type DemoFileServiceResponse
```

Initial value:

```
=  
| DemoFileCurrentResponse  
| {  
  kind: "file.empty";  
}
```

2.5.2.4 DemoFileTransferInput

```
export type DemoFileTransferInput
```

Initial value:

```
= {  
  name: string;  
  mimeType: string;  
  contentBase64: string;  
}
```

2.5.2.5 RegisteredServer

```
type RegisteredServer
```

Initial value:

```
= Awaited<ReturnType<typeof service.state> & {  
  serverId: string;  
  certificatePem: string;  
}
```

2.5.2.6 securityFlow

```
export const securityFlow
```

2.5.2.7 ServerDemoFileCurrentResponse

```
type ServerDemoFileCurrentResponse = Omit<DemoFileCurrentResponse, "encryptedPayload">
```


2.6 apps/client/src/lib/useSecurityFlow.ts File Reference

```
import useMutation;  
import "@tanstack/react-query";  
import "#/api";  
import "../client-security-state";  
import "../security-flow";
```

Functions

- function export [useSecurityFlow](#) ()

Variables

- const [CLIENT_STATUS_KEY](#) = ["client-status"] as const

2.6.1 Function Documentation

2.6.1.1 useSecurityFlow()

```
function export useSecurityFlow ( )
```

2.6.2 Variable Documentation

2.6.2.1 CLIENT_STATUS_KEY

```
const CLIENT_STATUS_KEY = ["client-status"] as const
```

2.7 apps/client/src/routes/index.tsx File Reference

```
import createFileRoute;  
import "@tanstack/react-router";  
import "lucide-react";  
import "react";  
import "#/components/evidence";  
import "#/components/ui/alert";  
import "#/components/ui/button";  
import "#/components/ui/card";  
import "#/components/ui/select";  
import "#/lib/security-flow";  
import "#/lib/useSecurityFlow";
```

Functions

- function [SecurityFlowPage](#) ()
- function [TransferredFilePreview](#) ({ file } : { file:DemoFileCurrentResponse })
- function [EncryptedPayloadView](#) ({ file } : { file:DemoFileCurrentResponse })
- function [readFileAsBase64](#) (file:File)
- function [isTextFile](#) (file:DemoFileCurrentResponse)
- function [base64ByteLength](#) (value:string)
- function [base64ToText](#) (value:string)

Variables

- type [FileServiceId](#) = "upload" | "view"
- export const [Route](#)

2.7.1 Function Documentation

2.7.1.1 base64ByteLength()

```
function base64ByteLength (
    value:string )
```

2.7.1.2 base64ToText()

```
function base64ToText (
    value:string )
```

2.7.1.3 EncryptedPayloadView()

```
function EncryptedPayloadView (
    { file } : { file:DemoFileCurrentResponse } )
```

2.7.1.4 isTextFile()

```
function isTextFile (
    file:DemoFileCurrentResponse )
```

2.7.1.5 readFileAsBase64()

```
function readFileAsBase64 (
    file:File )
```

2.7.1.6 SecurityFlowPage()

```
function SecurityFlowPage ( )
```

2.7.1.7 TransferredFilePreview()

```
function TransferredFilePreview (
    { file }: { file: DemoFileCurrentResponse } )
```

2.7.2 Variable Documentation

2.7.2.1 FileServiceId

```
type FileServiceId = "upload" | "view"
```

2.7.2.2 Route

```
export const Route
```

Initial value:

```
= createFileRoute("/") ({
    component: SecurityFlowPage,
})
```

2.8 apps/ttp/src/contract.ts File Reference

```
import hash;
import "@bsk/crypto";
```

Functions

- function export [serverAuthenticationPayload](#) (input: { serverId: string; certificatePem: string; requestId: string; userId: string; })
Builds the payload signed by the Server.
- function export [userAuthenticationPayload](#) (input: { userId: string; userCertificatePem: string; serverId: string; serverCertificatePem: string; requestId: string; })
Builds the canonical payload signed by the User.

Variables

- export [type Role](#) = "user" | "server"
- export [type IdentityPublicKeys](#)
- export [type RegisterIdentityInput](#)
- export [type ServerAuthenticationInput](#)
- export [type UserAuthenticationRequest](#)
- export [type UserAuthenticationInput](#)
- export [type ServerSessionKeyInput](#)
- export [type UserAuthRedirectInput](#)

2.8.1 Function Documentation

2.8.1.1 serverAuthenticationPayload()

```
function export serverAuthenticationPayload (
    input: { serverId: string; certificatePem: string; requestId: string; userId: string; } )
```

Builds the payload signed by the Server.

Parameters

<i>input</i>	Server identity, User identity, request id and certificate.
--------------	---

Returns

JSON string used for RSA signature verification.

2.8.1.2 userAuthenticationPayload()

```
function export userAuthenticationPayload (
    input:{ userId:string;userCertificatePem:string;serverId:string;serverCertificatePem:string;requestId:string;} )
```

Builds the canonical payload signed by the User.

Parameters

<i>input</i>	User, Server, request id and certificate material.
--------------	--

Returns

Stable JSON string used before session-key issuance.

2.8.2 Variable Documentation**2.8.2.1 IdentityPublicKeys**

```
export type IdentityPublicKeys
```

Initial value:

```
= {
  authPublicKeyPem: string;
  exchangePublicKeyPem: string;
}
```

2.8.2.2 RegisterIdentityInput

```
export type RegisterIdentityInput
```

Initial value:

```
= {
  role: Role;
  encryptedId: string;
  publicKey: IdentityPublicKeys;
}
```

2.8.2.3 Role

```
export type Role = "user" | "server"
```

2.8.2.4 ServerAuthenticationInput

```
export type ServerAuthenticationInput
```

Initial value:

```
= {  
  serverId: string;  
  certificatePem: string;  
  requestId: string;  
  userId: string;  
  signature: string;  
}
```

2.8.2.5 ServerSessionKeyInput

```
export type ServerSessionKeyInput
```

Initial value:

```
= {  
  requestId: string;  
}
```

2.8.2.6 UserAuthenticationInput

```
export type UserAuthenticationInput
```

Initial value:

```
= {  
  encryptedAuthMaterial: string;  
}
```

2.8.2.7 UserAuthenticationRequest

```
export type UserAuthenticationRequest
```

Initial value:

```
= {  
  userId: string;  
  userCertificatePem: string;  
  serverId: string;  
  serverCertificatePem: string;  
  requestId: string;  
  signature: string;  
}
```

2.8.2.8 UserAuthRedirectInput

```
export type UserAuthRedirectInput
```

Initial value:

```
= {  
  requestId: string;  
}
```

2.9 apps/server/src/protocol.ts File Reference

```
import aesGcm;
import "@bsk/crypto";
import "@bsk/rpc/client";
import "ttp";
import "ttp/contract";
import "../state";
```

Functions

- function async [registerProtectedServer](#) ()
Registers the protected Server with the TTP.
- function async [requestService](#) (input:{ userId:string })
Starts a protected service request by authenticating the Server to the TTP.
- function async [fetchServerSessionKey](#) (input:{ requestId:string })
Fetches and decrypts the Server session ticket.
- function export [closeLocalSession](#) ()
Clears the Server's local session state after service completion.
- function export [exchangeProtectedServiceData](#) (payload:SessionEncryptedPayload)
Handles one encrypted protected-service request.
- function [handleFileServiceRequest](#) (request:DemoFileServiceRequest)

Variables

- const [ttpBaseUrl](#) = process.env.TTP_API_BASE_URL ?? "http://localhost:3001"
- const [ttp](#) = createRpcClient<[TtpRouter](#)>([ttpBaseUrl](#))
- type [DemoFileServiceRequest](#)
- type [DemoFileResponse](#)
- type [DemoFileEmptyResponse](#)
- type [DemoFileServiceResponse](#) = [DemoFileResponse](#) | [DemoFileEmptyResponse](#)
- type [UploadFileRequest](#)

2.9.1 Function Documentation

2.9.1.1 closeLocalSession()

```
function export closeLocalSession ( )
```

Clears the Server's local session state after service completion.

Returns

Previous session id and updated Server status.

2.9.1.2 exchangeProtectedServiceData()

```
function export exchangeProtectedServiceData (
    payload:SessionEncryptedPayload )
```

Handles one encrypted protected-service request.

Parameters

<i>payload</i>	AES-GCM payload bound to the active session id.
----------------	---

Returns

Encrypted service response using the same session key.

2.9.1.3 fetchServerSessionKey()

```
function async fetchServerSessionKey (
    input:{ requestId:string } )
```

Fetches and decrypts the Server session ticket.

Parameters

<i>input</i>	Service request id whose User authentication has completed.
--------------	---

Returns

Updated Server status with an active session.

2.9.1.4 handleFileServiceRequest()

```
function handleFileServiceRequest (
    request:DemoFileServiceRequest )
```

2.9.1.5 registerProtectedServer()

```
function async registerProtectedServer ( )
```

Registers the protected Server with the TTP.

Returns

Current Server status including its issued certificate.

2.9.1.6 requestService()

```
function async requestService (
    input:{ userId:string } )
```

Starts a protected service request by authenticating the Server to the TTP.

Parameters

<i>input</i>	User id requesting the service.
--------------	---------------------------------

Returns

Request id used for the following User-authentication step.

2.9.2 Variable Documentation**2.9.2.1 DemoFileEmptyResponse**

```
type DemoFileEmptyResponse
```

Initial value:

```
= {
  kind: "file.empty";
}
```

2.9.2.2 DemoFileResponse

```
type DemoFileResponse
```

Initial value:

```
= {
  kind: "file.current";
  name: string;
  mimeType: string;
  size: number;
  contentBase64: string;
  storedAt: string;
}
```

2.9.2.3 DemoFileServiceRequest

```
type DemoFileServiceRequest
```

Initial value:

```
=
| {
  kind: "file.upload";
  name: string;
  mimeType: string;
  contentBase64: string;
}
| {
  kind: "file.view";
}
```

2.9.2.4 DemoFileServiceResponse

```
type DemoFileServiceResponse = DemoFileResponse | DemoFileEmptyResponse
```

2.9.2.5 ttp

```
const ttp = createRpcClient<TtpRouter>(ttpBaseUrl)
```


2.9.2.6 ttpBaseUrl

```
const ttpBaseUrl = process.env.TTP_API_BASE_URL ?? "http://localhost:3001"
```

2.9.2.7 UploadFileRequest

```
type UploadFileRequest
```

Initial value:

```
= {
  kind: "file.upload";
  name: string;
  mimeType: string;
  contentBase64: string;
}
```

2.10 apps/ttp/src/protocol.ts File Reference

```
import issueIdentityCertificate;
import "@bsk/crypto";
import "./contract";
import "./state";
```

Functions

- function export [ttpPublicKeyResponse](#) ()
- function export [registerIdentity](#) (input:RegisterIdentityInput)
Registers a User or Server and issues its identity certificate.
- function export [authenticateServerCertificate](#) (input:ServerAuthenticationInput)
Validates Server authentication and creates a pending User-auth request.
- function export [requestUserAuthentication](#) (input:UserAuthRedirectInput)
Returns the User-authentication redirect after Server validation.
- function export [authenticateUserForServer](#) (input:UserAuthenticationInput)
Validates User authentication and issues encrypted session tickets.
- function export [serverSessionKey](#) (input:ServerSessionKeyInput)
Returns the Server copy of the encrypted session ticket.
- function export [closeSession](#) (sessionId:string)
Marks a negotiated session as closed in the TTP state.
- function [decryptUserAuthenticationRequest](#) (encryptedAuthMaterial:string,)

Variables

- [type CertifiableIdentity](#) = Pick<IdentityRecord, "role" | "subjectId" | "publicKeys">
- [type CertificateClaim](#)

2.10.1 Function Documentation

2.10.1.1 authenticateServerCertificate()

```
function export authenticateServerCertificate (
  input:ServerAuthenticationInput )
```

Validates Server authentication and creates a pending User-auth request.

Parameters

<i>input</i>	Server certificate, request id, User id and Server signature.
--------------	---

Returns

Server-authentication confirmation for the request.

2.10.1.2 authenticateUserForServer()

```
function export authenticateUserForServer (
    input:UserAuthenticationInput )
```

Validates User authentication and issues encrypted session tickets.

Parameters

<i>input</i>	User authentication material encrypted to the TTP.
--------------	--

Returns

The validated request and encrypted User session ticket.

2.10.1.3 closeSession()

```
function export closeSession (
    sessionId:string )
```

Marks a negotiated session as closed in the TTP state.

Parameters

<i>session↔ Id</i>	Session identifier to close.
------------------------	------------------------------

Returns

Close confirmation with timestamp.

2.10.1.4 decryptUserAuthenticationRequest()

```
function decryptUserAuthenticationRequest (
    encryptedAuthMaterial:string )
```

2.10.1.5 registerIdentity()

```
function export registerIdentity (
    input: RegisterIdentityInput )
```

Registers a User or Server and issues its identity certificate.

Parameters

<i>input</i>	Encrypted subject id, role and public keys submitted by the identity.
--------------	---

Returns

Registered subject id, certificate and issuance timestamp.

2.10.1.6 requestUserAuthentication()

```
function export requestUserAuthentication (
    input: UserAuthRedirectInput )
```

Returns the User-authentication redirect after Server validation.

Parameters

<i>input</i>	Service request id.
--------------	---------------------

Returns

Redirect metadata telling the Client to submit User authentication.

2.10.1.7 serverSessionKey()

```
function export serverSessionKey (
    input: ServerSessionKeyInput )
```

Returns the Server copy of the encrypted session ticket.

Parameters

<i>input</i>	Completed service request id.
--------------	-------------------------------

Returns

Session id and ticket encrypted with the Server exchange key.

2.10.1.8 ttpPublicKeyResponse()

```
function export ttpPublicKeyResponse ( )
```

2.10.2 Variable Documentation

2.10.2.1 CertifiableIdentity

```
type CertifiableIdentity = Pick<IdentityRecord, "role" | "subjectId" | "publicKeys">
```

2.10.2.2 CertificateClaim

```
type CertificateClaim
```

Initial value:

```
= Pick<IdentityRecord, "role" | "subjectId"> & {
  certificatePem: string;
}
```

2.11 apps/server/src/rpc.ts File Reference

```
import type;
import "@bsk/crypto";
import "@orpc/server";
import "./protocol";
import "./state";
```

Functions

- function `reject` (code:"BAD_REQUEST"|"UNAUTHORIZED", error:unknown)

Variables

- export `type ServiceRouter` = typeof serviceRouter

2.11.1 Function Documentation

2.11.1.1 reject()

```
function reject (
  code:"BAD_REQUEST"|"UNAUTHORIZED",
  error:unknown )
```

2.11.2 Variable Documentation

2.11.2.1 ServiceRouter

```
export type ServiceRouter = typeof serviceRouter
```

2.12 apps/ttp/src/rpc.ts File Reference

```
import RSA_BITS;
import "@bsk/crypto";
import "@orpc/server";
import "../contract";
import "../protocol";
import "../state";
```

Functions

- function [reject](#) (code:"BAD_REQUEST"|"UNAUTHORIZED"|"NOT_FOUND", error:unknown)

Variables

- export [type TtpRouter](#) = typeof ttpRouter

2.12.1 Function Documentation

2.12.1.1 [reject\(\)](#)

```
function reject (
    code:"BAD_REQUEST"|"UNAUTHORIZED"|"NOT_FOUND",
    error:unknown )
```

2.12.2 Variable Documentation

2.12.2.1 [TtpRouter](#)

```
export type TtpRouter = typeof ttpRouter
```

2.13 apps/server/src/state.ts File Reference

```
import type;
import "@bsk/crypto";
import "@bsk/rpc/log";
```

Functions

- function export [log](#) (event:string, details:string, level:SecurityLogLevel="info")
- function export [resetServiceServerState](#) ()
- function export [readServiceServerStatus](#) ()

Variables

- [type ServiceServerState](#)
- export interface [ServiceServerStatus](#)
- export const [state](#)
- export const export [type DemoFileMeta](#)
- export [type DemoStoredFile](#)
- const [logger](#) = [createSecurityLogger](#)(["application.log", "server.log"])

2.13.1 Function Documentation

2.13.1.1 log()

```
function export log (
    event:string,
    details:string,
    level:SecurityLogLevel = "info" )
```

2.13.1.2 readServiceServerStatus()

```
function export readServiceServerStatus ( )
```

2.13.1.3 resetServiceServerState()

```
function export resetServiceServerState ( )
```

2.13.2 Variable Documentation

2.13.2.1 DemoFileMeta

```
export const export type DemoFileMeta
```

Initial value:

```
= {
  name: string;
  size: number;
  mimeType: string;
}
```

2.13.2.2 DemoStoredFile

```
export type DemoStoredFile
```

Initial value:

```
= DemoFileMeta & {
  contentBase64: string;
  storedAt: string;
}
```

2.13.2.3 logger

```
const logger = createSecurityLogger(["application.log", "server.log"])
```

2.13.2.4 ServiceServerState

```
type ServiceServerState
```

Initial value:

```
= {
  serverId?: string;
  authKeyPair?: RsaPair;
  exchangeKeyPair?: RsaPair;
  certificatePem?: string;
  sessionId?: string;
  sessionKey?: string;
  serviceExchanged?: boolean;
  latestDemoFile?: DemoStoredFile;
  lastServiceEvent?: { event: string; details: string };
  pendingRequests: Map<string, { userId: string; createdAt: string }>;
}
```

2.13.2.5 ServiceServerStatus

```
export interface ServiceServerStatus
```

Initial value:

```
{
  registered: boolean
}
```

2.13.2.6 state

```
export const state
```

2.14 apps/ttp/src/state.ts File Reference

```
import createCertificateAuthority;
import "@bsk/crypto";
import "@bsk/rpc/log";
import "./contract";
```

Variables

- export interface [IdentityRecord](#)
- export interface [SessionRecord](#)
- export interface [PendingAuthRecord](#)
- export const [ca](#)
- export const [registeredIdentities](#) = new Map<string, [IdentityRecord](#)>()
- export const [sessions](#) = new Map<string, [SessionRecord](#)>()
- export const [sessionsByRequest](#) = new Map<string, string>()
- export const [pendingAuths](#) = new Map<string, [PendingAuthRecord](#)>()
- const [logger](#) = [createSecurityLogger](#)(["application.log", "ttp.log"])
- export const [log](#) = logger.log

2.14.1 Variable Documentation

2.14.1.1 ca

```
export const ca
```

Initial value:

```
= await createCertificateAuthority({  
  commonName: "BSK PG Trusted Third Party",  
  organization: "BSK PG Demo",  
})
```

2.14.1.2 IdentityRecord

```
export interface IdentityRecord
```

Initial value:

```
{  
  role: Role
```

2.14.1.3 log

```
export const log = logger.log
```

2.14.1.4 logger

```
const logger = createSecurityLogger(["application.log", "ttp.log"])
```

2.14.1.5 PendingAuthRecord

```
export interface PendingAuthRecord
```

Initial value:

```
{  
  requestId: string
```

2.14.1.6 pendingAuths

```
export const pendingAuths = new Map<string, PendingAuthRecord>()
```

2.14.1.7 registeredIdentities

```
export const registeredIdentities = new Map<string, IdentityRecord>()
```

2.14.1.8 SessionRecord

```
export interface SessionRecord
```

Initial value:

```
{  
  sessionId: string
```


2.14.1.9 sessions

```
export const sessions = new Map<string, SessionRecord>()
```

2.14.1.10 sessionsByRequest

```
export const sessionsByRequest = new Map<string, string>()
```

2.15 packages/crypto/src/certificates.ts File Reference

```
import forge;  
import "node-forge";  
import "./crypto";  
import "./types";
```

Functions

- function async [createCertificateAuthority](#) (options:CertificateAuthorityOptions,) *Creates the self-signed TTP certificate authority.*

Variables

- const [DAY_MS](#) = 24 * 60 * 60 * 1000

2.15.1 Function Documentation

2.15.1.1 createCertificateAuthority()

```
function async createCertificateAuthority (  
    options:CertificateAuthorityOptions )
```

Creates the self-signed TTP certificate authority.

Parameters

<i>options</i>	Certificate subject and validity settings.
----------------	--

Returns

PEM keys and the forge certificate object used for signing identity certificates. Issues a TTP-signed X.509 identity certificate.

Parameters

<i>input</i>	Registered identity, exchange public key and issuing authority.
--------------	---

Returns

PEM-encoded X.509 certificate for the User or Server exchange key. Verifies that a certificate chains to the trusted TTP authority.

Parameters

<i>authority</i>	Trusted TTP certificate authority.
<i>certificatePem</i>	PEM-encoded certificate to validate.

Returns

The validated certificate common name.

2.15.2 Variable Documentation**2.15.2.1 DAY_MS**

```
const DAY_MS = 24 * 60 * 60 * 1000
```

2.16 packages/crypto/src/crypto.ts File Reference

```
import forge;
import "node-forge";
import "../types";
```

Functions

- function [rsaOaepSha256](#) ()
- function [encryptRsaOaepSha256](#) (publicKey:forge.pki.rsa.PublicKey, plaintext:string)
- function [encodeHybridPayloadBase64](#) (payload:HybridEncryptedPayload)

Variables

- export const [RSA_BITS](#) = 4096
RSA modulus size required by the project specification.
- const [AES_GCM_TAG_BITS](#) = 128
AES-GCM tag length used for authenticated session payloads.
- const [AES_GCM_IV_BYTES](#) = 12
- const [AES_256_KEY_BYTES](#) = 32
AES-256 session key length in bytes.
- export const [random](#)
- export const [signedPayload](#)
- export const [aesGcm](#)
- export const [rsa](#)

2.16.1 Function Documentation

2.16.1.1 encodeHybridPayloadBase64()

```
function encodeHybridPayloadBase64 (
    payload:HybridEncryptedPayload )
```

Creates SHA-256 identifiers and short fingerprints for certificates or UI evidence.

2.16.1.2 encryptRsaOaepSha256()

```
function encryptRsaOaepSha256 (
    publicKey:forge.pki.rsa. PublicKey,
    plaintext:string )
```

2.16.1.3 rsaOaepSha256()

```
function rsaOaepSha256 ( )
```

2.16.2 Variable Documentation

2.16.2.1 AES_256_KEY_BYTES

```
const AES_256_KEY_BYTES = 32
```

AES-256 session key length in bytes.

2.16.2.2 AES_GCM_IV_BYTES

```
const AES_GCM_IV_BYTES = 12
```

2.16.2.3 AES_GCM_TAG_BITS

```
const AES_GCM_TAG_BITS = 128
```

AES-GCM tag length used for authenticated session payloads.

2.16.2.4 aesGcm

```
export const aesGcm
```

2.16.2.5 random

```
export const random
```

Initial value:

```
= {  
  
  hex(bytes: number): string {  
    const value = forge.random.getBytesSync(bytes);  
    return forge.util.bytesToHex(value);  
  },  
  
  uuid() {  
    return crypto.randomUUID();  
  },  
  
  sessionKey(): string {  
    const key = forge.random.getBytesSync(AES_256_KEY_BYTES);  
    return forge.util.encode64(key);  
  },  
}
```

2.16.2.6 rsa

```
export const rsa
```

2.16.2.7 RSA_BITS

```
export const RSA_BITS = 4096
```

RSA modulus size required by the project specification.

2.16.2.8 signedPayload

```
export const signedPayload
```

Initial value:

```
= {  
  
  from(fields: Record<string, string>): string {  
    const sortedFields = Object.fromEntries(  
      Object.entries(fields).sort(([left], [right]) => left.localeCompare(right)),  
    );  
    return JSON.stringify(sortedFields);  
  },  
}
```

2.17 packages/crypto/src/types.ts File Reference

```
import type;  
import "node-forge";
```

Variables

- export type [RsaPair](#)
- export interface [RsaEncryptor](#)
- verify(plaintext:string, signatureBase64:string) export interface [RsaDecryptor](#)
Verifies a base64 RSA/SHA-256 signature for the given plaintext.
- sign(plaintext:string) export type [AesGcmPayload](#)
Signs the given plaintext with RSA/SHA-256 and returns a base64 signature.
- export type [SessionEncryptedPayload](#)
- export type [SessionTicket](#)
- export type [HybridEncryptedPayload](#)
- export type [CertificateAuthorityOptions](#)
- export type [CertificateAuthority](#)
- export type [IdentityCertificateInput](#)

2.17.1 Variable Documentation

2.17.1.1 AesGcmPayload

```
sign (plaintext: string) export type AesGcmPayload
```

Initial value:

```
= {  
  iv: string;  
  ciphertext: string;  
  authTag: string;  
}
```

Signs the given plaintext with RSA/SHA-256 and returns a base64 signature.

2.17.1.2 CertificateAuthority

```
export type CertificateAuthority
```

Initial value:

```
= {  
  privateKey: forge.pki.rsa.PrivateKey;  
  certificate: forge.pki.Certificate;  
  privateKeyPem: string;  
  publicKeyPem: string;  
  certificatePem: string;  
}
```

2.17.1.3 CertificateAuthorityOptions

```
export type CertificateAuthorityOptions
```

Initial value:

```
= {  
  commonName: string;  
  organization: string;  
  validDays?: number;  
}
```

2.17.1.4 HybridEncryptedPayload

```
export type HybridEncryptedPayload
```

Initial value:

```
= AesGcmPayload & {  
  encryptedKey: string;  
}
```

2.17.1.5 IdentityCertificateInput

```
export type IdentityCertificateInput
```

Initial value:

```
= {  
  authority: CertificateAuthority;  
  role: string;  
  subjectId: string;  
  publicKeyPem: string;  
  organization?: string;  
  validDays?: number;  
}
```

2.17.1.6 RsaDecryptor

```
verify (plaintext: string, signatureBase64: string) export interface RsaDecryptor
```

Initial value:

```
{  
  
  decrypt(payloadBase64: string): string
```

Verifies a base64 RSA/SHA-256 signature for the given plaintext.

2.17.1.7 RsaEncryptor

```
export interface RsaEncryptor
```

Initial value:

```
{  
  
  encrypt(plaintext: string): string
```

2.17.1.8 RsaPair

```
export type RsaPair
```

Initial value:

```
= {  
  publicKeyPem: string;  
  privateKeyPem: string;  
}
```

2.17.1.9 SessionEncryptedPayload

```
export type SessionEncryptedPayload
```

Initial value:

```
= AesGcmPayload & {  
  sessionId: string;  
}
```

2.17.1.10 SessionTicket

```
export type SessionTicket
```

Initial value:

```
= {  
  sessionId: string;  
  sessionKey: string;  
}
```

2.18 packages/rpc/src/client.ts File Reference

```
import createORPCClient;  
import "@orpc/client";  
import "@orpc/client/fetch";  
import "@orpc/server";
```

2.19 packages/rpc/src/log.ts File Reference

```
import truncateSync;  
import "node:fs";  
import "node:path";  
import "pino";  
import "pino-pretty";
```

Functions

- function `logDirectory` ()
- function `logPath` (fileName:string)
- function export `createSecurityLogger` (fileNames:readonly string[])

Variables

- export type `SecurityLogActor` = "ttp" | "user" | "server"
- export type `SecurityLogLevel` = "info" | "warn" | "error"
- const `DEFAULT_LOG_DIR` = "../logs"

2.19.1 Function Documentation

2.19.1.1 createSecurityLogger()

```
function export createSecurityLogger (
    fileNames:readonly string[] )
```

2.19.1.2 logDirectory()

```
function logDirectory ( )
```

2.19.1.3 logPath()

```
function logPath (
    fileName:string )
```

2.19.2 Variable Documentation

2.19.2.1 DEFAULT_LOG_DIR

```
const DEFAULT_LOG_DIR = "../../logs"
```

2.19.2.2 SecurityLogActor

```
export type SecurityLogActor = "ttp" | "user" | "server"
```

2.19.2.3 SecurityLogLevel

```
export type SecurityLogLevel = "info" | "warn" | "error"
```

2.20 packages/rpc/src/server.ts File Reference

```
import type;
import "@orpc/server";
import "@orpc/server/fetch";
import "@orpc/server/plugins";
```

Functions

- function export [createRpcFetch](#) (router:AnyRouter, banner:string)

2.20.1 Function Documentation

2.20.1.1 createRpcFetch()

```
function export createRpcFetch (
    router:AnyRouter,
    banner:string )
```


Index

- ActiveSession
 - client-security-state.ts, [6](#)
- activeSession
 - security-flow.ts, [7](#)
- AES_256_KEY_BYTES
 - crypto.ts, [27](#)
- AES_GCM_IV_BYTES
 - crypto.ts, [27](#)
- AES_GCM_TAG_BITS
 - crypto.ts, [27](#)
- aesGcm
 - crypto.ts, [27](#)
- AesGcmPayload
 - types.ts, [29](#)
- api.ts
 - encryptForTtp, [3](#)
 - loadTtpPublicKey, [3](#)
 - service, [4](#)
 - serviceBaseUrl, [4](#)
 - serviceQuery, [4](#)
 - ttp, [4](#)
 - ttpBaseUrl, [4](#)
 - ttpProtocol, [4](#)
 - ttpQuery, [4](#)
 - type, [5](#)
- apps/client/src/api.ts, [3](#)
- apps/client/src/components/evidence.tsx, [5](#)
- apps/client/src/components/utils.ts, [5](#)
- apps/client/src/lib/client-security-state.ts, [5](#)
- apps/client/src/lib/security-flow.ts, [6](#)
- apps/client/src/lib/useSecurityFlow.ts, [9](#)
- apps/client/src/routes/index.tsx, [9](#)
- apps/server/src/protocol.ts, [14](#)
- apps/server/src/rpc.ts, [20](#)
- apps/server/src/state.ts, [21](#)
- apps/ttp/src/contract.ts, [11](#)
- apps/ttp/src/protocol.ts, [17](#)
- apps/ttp/src/rpc.ts, [21](#)
- apps/ttp/src/state.ts, [23](#)
- authenticateServerCertificate
 - protocol.ts, [17](#)
- authenticateUserForServer
 - protocol.ts, [18](#)
- base64ByteLength
 - index.tsx, [10](#)
- base64ToText
 - index.tsx, [10](#)
- ca
 - state.ts, [24](#)
- CertiifiableIdentity
 - protocol.ts, [20](#)
- CertificateAuthority
 - types.ts, [29](#)
- CertificateAuthorityOptions
 - types.ts, [29](#)
- CertificateClaim
 - protocol.ts, [20](#)
- certificates.ts
 - createCertificateAuthority, [25](#)
 - DAY_MS, [26](#)
- client-security-state.ts
 - ActiveSession, [6](#)
 - clientSecurityState, [6](#)
 - current, [6](#)
 - RegisteredUser, [6](#)
- CLIENT_STATUS_KEY
 - useSecurityFlow.ts, [9](#)
- clientSecurityState
 - client-security-state.ts, [6](#)
- closeLocalSession
 - protocol.ts, [14](#)
- closeSession
 - protocol.ts, [18](#)
- cn
 - utils.ts, [5](#)
- contract.ts
 - IdentityPublicKeys, [12](#)
 - RegisterIdentityInput, [12](#)
 - Role, [12](#)
 - ServerAuthenticationInput, [12](#)
 - serverAuthenticationPayload, [11](#)
 - ServerSessionKeyInput, [13](#)
 - UserAuthenticationInput, [13](#)
 - userAuthenticationPayload, [12](#)
 - UserAuthenticationRequest, [13](#)
 - UserAuthRedirectInput, [13](#)
- createCertificateAuthority
 - certificates.ts, [25](#)
- createRogueUserCertificate
 - security-flow.ts, [7](#)
- createRpcFetch
 - server.ts, [32](#)
- createSecurityLogger
 - log.ts, [32](#)
- crypto.ts
 - AES_256_KEY_BYTES, [27](#)
 - AES_GCM_IV_BYTES, [27](#)

- AES_GCM_TAG_BITS, 27
- aesGcm, 27
- encodeHybridPayloadBase64, 27
- encryptRsaOaepSha256, 27
- random, 27
- rsa, 28
- RSA_BITS, 28
- rsaOaepSha256, 27
- signedPayload, 28
- current
 - client-security-state.ts, 6
- DAY_MS
 - certificates.ts, 26
- decryptUserAuthenticationRequest
 - protocol.ts, 18
- DEFAULT_LOG_DIR
 - log.ts, 32
- DemoFileCurrentResponse
 - security-flow.ts, 7
- DemoFileEmptyResponse
 - protocol.ts, 16
- DemoFileMeta
 - state.ts, 22
- DemoFileResponse
 - protocol.ts, 16
- DemoFileServiceRequest
 - protocol.ts, 16
 - security-flow.ts, 7
- DemoFileServiceResponse
 - protocol.ts, 16
 - security-flow.ts, 8
- DemoFileTransferInput
 - security-flow.ts, 8
- DemoStoredFile
 - state.ts, 22
- encodeHybridPayloadBase64
 - crypto.ts, 27
- EncryptedPayloadView
 - index.tsx, 10
- encryptForTtp
 - api.ts, 3
- encryptRsaOaepSha256
 - crypto.ts, 27
- Evidence
 - evidence.tsx, 5
- evidence.tsx
 - Evidence, 5
- exchangeProtectedServiceData
 - protocol.ts, 14
- fetchServerSessionKey
 - protocol.ts, 15
- FileServiceId
 - index.tsx, 11
- handleFileServiceRequest
 - protocol.ts, 15
- HybridEncryptedPayload
 - types.ts, 29
- IdentityCertificateInput
 - types.ts, 30
- IdentityPublicKeys
 - contract.ts, 12
- IdentityRecord
 - state.ts, 24
- index.tsx
 - base64ByteLength, 10
 - base64ToText, 10
 - EncryptedPayloadView, 10
 - FileServiceId, 11
 - isTextFile, 10
 - readFileAsBase64, 10
 - Route, 11
 - SecurityFlowPage, 10
 - TransferredFilePreview, 10
- isTextFile
 - index.tsx, 10
- loadRegisteredServer
 - security-flow.ts, 7
- loadTtpPublicKey
 - api.ts, 3
- log
 - state.ts, 22, 24
- log.ts
 - createSecurityLogger, 32
 - DEFAULT_LOG_DIR, 32
 - logDirectory, 32
 - logPath, 32
 - SecurityLogActor, 32
 - SecurityLogLevel, 32
- logDirectory
 - log.ts, 32
- logger
 - state.ts, 22, 24
- logPath
 - log.ts, 32
- packages/crypto/src/certificates.ts, 25
- packages/crypto/src/crypto.ts, 26
- packages/crypto/src/types.ts, 28
- packages/rpc/src/client.ts, 31
- packages/rpc/src/log.ts, 31
- packages/rpc/src/server.ts, 32
- PendingAuthRecord
 - state.ts, 24
- pendingAuths
 - state.ts, 24
- protocol.ts
 - authenticateServerCertificate, 17
 - authenticateUserForServer, 18
 - CertifiableIdentity, 20
 - CertificateClaim, 20
 - closeLocalSession, 14
 - closeSession, 18

- decryptUserAuthenticationRequest, 18
- DemoFileEmptyResponse, 16
- DemoFileResponse, 16
- DemoFileServiceRequest, 16
- DemoFileServiceResponse, 16
- exchangeProtectedServiceData, 14
- fetchServerSessionKey, 15
- handleFileServiceRequest, 15
- registerIdentity, 18
- registerProtectedServer, 15
- requestService, 15
- requestUserAuthentication, 19
- serverSessionKey, 19
- ttp, 16
- ttpBaseUrl, 16
- ttpPublicKeyResponse, 19
- UploadFileRequest, 17
- random
 - crypto.ts, 27
- readFileAsBase64
 - index.tsx, 10
- readServiceServerStatus
 - state.ts, 22
- registeredIdentities
 - state.ts, 24
- RegisteredServer
 - security-flow.ts, 8
- RegisteredUser
 - client-security-state.ts, 6
- registerIdentity
 - protocol.ts, 18
- RegisterIdentityInput
 - contract.ts, 12
- registerProtectedServer
 - protocol.ts, 15
- registerUserIdentity
 - security-flow.ts, 7
- reject
 - rpc.ts, 20, 21
- requestService
 - protocol.ts, 15
- requestUserAuthentication
 - protocol.ts, 19
- resetServiceServerState
 - state.ts, 22
- Role
 - contract.ts, 12
- Route
 - index.tsx, 11
- rpc.ts
 - reject, 20, 21
 - ServiceRouter, 20
 - TtpRouter, 21
- rsa
 - crypto.ts, 28
- RSA_BITS
 - crypto.ts, 28
- RsaDecryptor
 - types.ts, 30
- RsaEncryptor
 - types.ts, 30
- rsaOaepSha256
 - crypto.ts, 27
- RsaPair
 - types.ts, 30
- security-flow.ts
 - activeSession, 7
 - createRogueUserCertificate, 7
 - DemoFileCurrentResponse, 7
 - DemoFileServiceRequest, 7
 - DemoFileServiceResponse, 8
 - DemoFileTransferInput, 8
 - loadRegisteredServer, 7
 - RegisteredServer, 8
 - registerUserIdentity, 7
 - securityFlow, 8
 - ServerDemoFileCurrentResponse, 8
- securityFlow
 - security-flow.ts, 8
- SecurityFlowPage
 - index.tsx, 10
- SecurityLogActor
 - log.ts, 32
- SecurityLogLevel
 - log.ts, 32
- server.ts
 - createRpcFetch, 32
- ServerAuthenticationInput
 - contract.ts, 12
- serverAuthenticationPayload
 - contract.ts, 11
- ServerDemoFileCurrentResponse
 - security-flow.ts, 8
- serverSessionKey
 - protocol.ts, 19
- ServerSessionKeyInput
 - contract.ts, 13
- service
 - api.ts, 4
- serviceBaseUrl
 - api.ts, 4
- serviceQuery
 - api.ts, 4
- ServiceRouter
 - rpc.ts, 20
- ServiceServerState
 - state.ts, 23
- ServiceServerStatus
 - state.ts, 23
- SessionEncryptedPayload
 - types.ts, 30
- SessionRecord
 - state.ts, 24
- sessions
 - state.ts, 24
- sessionsByRequest

- state.ts, [25](#)
- SessionTicket
 - types.ts, [31](#)
- signedPayload
 - crypto.ts, [28](#)
- state
 - state.ts, [23](#)
- state.ts
 - ca, [24](#)
 - DemoFileMeta, [22](#)
 - DemoStoredFile, [22](#)
 - IdentityRecord, [24](#)
 - log, [22](#), [24](#)
 - logger, [22](#), [24](#)
 - PendingAuthRecord, [24](#)
 - pendingAuths, [24](#)
 - readServiceServerStatus, [22](#)
 - registeredIdentities, [24](#)
 - resetServiceServerState, [22](#)
 - ServiceServerState, [23](#)
 - ServiceServerStatus, [23](#)
 - SessionRecord, [24](#)
 - sessions, [24](#)
 - sessionsByRequest, [25](#)
 - state, [23](#)
- TransferredFilePreview
 - index.tsx, [10](#)
- ttp
 - api.ts, [4](#)
 - protocol.ts, [16](#)
- ttpBaseUrl
 - api.ts, [4](#)
 - protocol.ts, [16](#)
- ttpProtocol
 - api.ts, [4](#)
- ttpPublicKeyResponse
 - protocol.ts, [19](#)
- ttpQuery
 - api.ts, [4](#)
- TtpRouter
 - rpc.ts, [21](#)
- type
 - api.ts, [5](#)
- types.ts
 - AesGcmPayload, [29](#)
 - CertificateAuthority, [29](#)
 - CertificateAuthorityOptions, [29](#)
 - HybridEncryptedPayload, [29](#)
 - IdentityCertificateInput, [30](#)
 - RsaDecryptor, [30](#)
 - RsaEncryptor, [30](#)
 - RsaPair, [30](#)
 - SessionEncryptedPayload, [30](#)
 - SessionTicket, [31](#)
- UploadFileRequest
 - protocol.ts, [17](#)
- UserAuthenticationInput
 - contract.ts, [13](#)
- userAuthenticationPayload
 - contract.ts, [12](#)
- UserAuthenticationRequest
 - contract.ts, [13](#)
- UserAuthRedirectInput
 - contract.ts, [13](#)
- useSecurityFlow
 - useSecurityFlow.ts, [9](#)
- useSecurityFlow.ts
 - CLIENT_STATUS_KEY, [9](#)
 - useSecurityFlow, [9](#)
- utils.ts
 - cn, [5](#)